IIOT EXTERNAL PRACTICAL – COMPLETE PREPARATION GUIDE

All 5 Lab Experiments | ADYPSOE | Computer Laboratory II: Industrial Internet of Things

Purpose

This document consolidates the five experiment walkthroughs prepared in this chat, including aims, requirements, circuit connections, step-by-step workflows, programs, code explanations, expected outputs, troubleshooting, conclusions, viva questions, checklists, and revision notes. The original lab manual is the primary reference. Where its extracted text did not provide complete code, a complete working example was supplied in the walkthrough and is identified as such.

CONTENTS

1. Experiment 1 – Sending Alert Messages Using ESP32 and Potentiometer

2. Experiment 2 – Interfacing PIR Sensor with Arduino using Tinkercad

3. Experiment 3 – CPS Monitoring and Controlling using ESP32 Web Server

4. Experiment 4 – Sending Sensor Data and Displaying it on a Webpage

5. Experiment 5 – Industrial Data Analysis using Excel

6. Final revision sheet and preparation strategy

EXPERIMENT 1 – SENDING ALERT MESSAGES USING ESP32 AND POTENTIOMETER

Lab manual experiment 4.1 | Arduino IDE + ESP32

1. AIM

Write a program for sending alert messages to the user for controlling and interacting with the environment.

What you are doing:

Connect a 10K potentiometer to an ESP32, read its changing analog values, display them on the Serial Monitor, and print a warning whenever the value exceeds 2000.

2. REQUIREMENTS

Software:

• Arduino IDE

Hardware:

• ESP32 (ESP-WROOM-32)

• 10K potentiometer

• Breadboard

• Micro USB data cable

• Male-to-female jumper wires

• Laptop/PC

3. CIRCUIT CONNECTIONS

Potentiometer pin → ESP32 connection:

• Middle pin → GPIO 34

• One outer pin (red in the manual) → 3.3V

• Other outer pin (orange in the manual) → GND

Remember:

• Middle → GPIO 34

• Red → 3.3V

• Orange → GND

The two outer pins can be swapped; this reverses the direction in which the readings increase. Use 3.3V, not 5V, for the ESP32 potentiometer circuit. GPIO 34 is an input-only, ADC-capable pin.

4. EXACT PRACTICAL WORKFLOW

Step 1 – Connect ESP32 to laptop

1. Take the ESP32 board.

2. Connect the Micro USB data cable to the ESP32.

3. Connect the other end to the laptop.

4. Check whether the power indicator LED glows.

Step 2 – Make circuit connections

1. Place the potentiometer on the breadboard.

2. Connect its middle pin to GPIO 34.

3. Connect one outer pin to 3.3V.

4. Connect the remaining outer pin to GND.

5. Double-check the connections.

Step 3 – Open and configure Arduino IDE

1. Open Arduino IDE.

2. Go to Tools → Board → ESP32 Arduino.

3. Select the ESP32 board, typically ESP32 Dev Module.

4. Go to Tools → Port.

5. Select the COM port where the ESP32 is connected.

If the COM port is not visible:

• Check that the USB cable supports data transfer.

• Reconnect the ESP32.

• Check Device Manager → Ports (COM & LPT).

• Install the appropriate USB-to-serial driver if required. The manual mentions CH340G, but the required driver depends on the USB chip fitted to the board.

Step 4 – Write the Arduino program

```
const int potPin = 34;
```

int potValue = 0;

```
void setup() {
```

Serial.begin(115200);

delay(1000);

```
}
void loop() {
```

potValue = analogRead(potPin);

```
  if (potValue > 2000) {
```

Serial.println("warning");

```
  }
```

Serial.println(potValue);

delay(500);

```
}
```

Step 5 – Verify and upload

1. Click Verify (checkmark).

2. Wait for compilation to finish.

3. If there are no errors, click Upload (right arrow).

4. Wait until uploading finishes.

5. If the IDE displays Connecting and gets stuck, hold the ESP32 BOOT button during connection, then release it when uploading begins.

6. Press the ESP32 RESET/EN button after uploading if required.

Step 6 – Open Serial Monitor

1. Go to Tools → Serial Monitor.

2. Set baud rate to 115200.

3. Press the ESP32 RESET/EN button.

4. Observe the potentiometer readings.

Step 7 – Test the potentiometer

1. Rotate the potentiometer slowly.

2. Observe the changing values.

3. When the value exceeds 2000, the program prints “warning”.

4. When the value is below or equal to 2000, it prints only the analog value.

5. CODE EXPLANATION

• const int potPin = 34; – Defines GPIO 34 as the input pin.

• int potValue = 0; – Stores the potentiometer reading.

• void setup() – Executes only once when the board starts or resets.

• Serial.begin(115200); – Starts serial communication at 115200 baud.

• delay(1000); – Waits for one second during initialization.

• void loop() – Runs continuously.

• analogRead(potPin) – Reads the analog input from GPIO 34.

• if (potValue > 2000) – Checks whether the reading exceeds 2000.

• Serial.println("warning"); – Prints the warning message.

• Serial.println(potValue); – Prints the current reading.

• delay(500); – Waits 500 milliseconds before the next iteration.

6. EXPECTED OUTPUT

Example Serial Monitor output:

1250

1450

1800

warning

2200

warning

2800

warning

3500

The values are examples; actual readings depend on the potentiometer position. The ESP32 analog reading is approximately 0 to 4095 (12-bit ADC).

Case 1 – Value below or equal to 2000:

Example reading: 1250

No warning is printed; the analog value is displayed.

Case 2 – Value above 2000:

Example reading: 2800

The warning message and analog value are printed.

7. EXPECTED RESULT

• ESP32 successfully reads the potentiometer’s analog value.

• Values range approximately from 0 to 4095.

• When the reading exceeds 2000, the program prints “warning”.

• Readings are displayed every 500 milliseconds.

8. CONCLUSION

Thus, we successfully interfaced a 10K potentiometer with ESP32 and developed a program to read analog values and generate an alert message whenever the value exceeds 2000.

9. COMMON ERRORS AND SOLUTIONS

• Port not showing – Check USB cable and driver.

• Failed to connect to ESP32 – Hold BOOT while uploading.

• No readings in Serial Monitor – Select 115200 baud.

• Reading always 0 – Check GPIO 34 and GND connections.

• Reading not changing – Check potentiometer middle-pin connection.

• Compilation error – Check board selection and code syntax.

10. IMPORTANT VIVA QUESTIONS AND ANSWERS

Q1. What is ESP32?

ESP32 is a low-cost microcontroller developed by Espressif Systems. It supports Wi-Fi, Bluetooth and GPIO interfacing.

Q2. What is a potentiometer?

A potentiometer is a variable resistor used to adjust resistance and produce a variable output voltage.

Q3. What is GPIO?

GPIO stands for General Purpose Input/Output. It allows a microcontroller to interact with external components.

Q4. Why did we use GPIO 34?

GPIO 34 is an ADC-capable input pin on ESP32 that can read analog signals. It is input-only.

Q5. What is ADC?

ADC stands for Analog-to-Digital Converter. It converts analog voltage into digital values.

Q6. Why is the maximum reading 4095?

ESP32 uses a 12-bit ADC reading by default in this experiment. 2^12 = 4096 possible values, so the range is 0 to 4095.

Q7. What is Serial Monitor?

It is a tool in Arduino IDE used to display communication and debugging messages from the microcontroller.

Q8. What is baud rate?

Baud rate is the speed of serial communication. Here, 115200 bits per second is selected.

Q9. What is the purpose of delay(500)?

It pauses the program for 500 milliseconds between readings.

Q10. What happens when the potentiometer value exceeds 2000?

The condition becomes true, and ESP32 prints “warning” on the Serial Monitor.

Q11. Difference between analogRead() and digitalRead()?

analogRead() reads analog values; digitalRead() reads a digital HIGH or LOW state.

Q12. Why do we use 3.3V?

ESP32 GPIO uses 3.3V logic. Supplying the potentiometer with 3.3V keeps its analog output within a safe range.

11. EXECUTION CHECKLIST

■ Connect ESP32 to laptop using USB data cable.

■ Connect potentiometer middle pin to GPIO 34.

■ Connect outer pins to 3.3V and GND.

■ Open Arduino IDE and select ESP32 board.

■ Select correct COM port.

■ Paste the program.

■ Click Verify and check compilation.

■ Upload the program.

■ Open Serial Monitor at 115200 baud.

■ Rotate potentiometer and demonstrate warning above 2000.

12. QUICK REVISION

• Board: ESP32

• Sensor/input: 10K potentiometer

• Analog pin: GPIO 34

• Supply: 3.3V and GND

• ADC range: 0–4095

• Warning threshold: greater than 2000

• Serial baud rate: 115200

• Delay: 500 ms

EXPERIMENT 2 – INTERFACING PIR SENSOR WITH ARDUINO USING TINKERCAD

Lab manual experiment 4.2 | Tinkercad + Arduino Uno

1. AIM

Write an Arduino/Raspberry Pi program for interfacing with a PIR sensor.

What you are doing:

Connect a PIR motion sensor to Arduino Uno, detect human movement, turn an LED ON when motion is detected, turn it OFF when there is no motion, and display the status in the Serial Monitor.

2. REQUIREMENTS

Software:

• Tinkercad Circuits

Components:

• Arduino Uno R3

• PIR sensor

• LED

• Breadboard

• Jumper wires

• 220Ω resistor (recommended for the external LED)

3. UNDERSTAND THE PIR SENSOR

PIR stands for Passive Infrared Sensor. It detects changes in infrared radiation caused by moving objects, particularly humans and animals.

PIR pins:

• VCC – Power supply

• GND – Ground

• OUT / Signal – Digital output

Working:

• No motion detected → Output LOW.

• Motion detected → Output HIGH.

• Arduino reads this signal and controls the LED accordingly.

4. CIRCUIT CONNECTIONS

PIR sensor:

• VCC → Arduino 5V

• GND → Arduino GND

• OUT → Arduino digital pin 2

LED:

• LED positive/anode (long leg) → Digital pin 13 through a 220Ω resistor.

• LED negative/cathode (short leg) → GND.

Remember:

• PIR → 5V, GND, D2.

• LED → D13 through resistor, then GND.

The LED can use another output pin, but the code and circuit must use the same pin number.

5. EXACT PRACTICAL WORKFLOW

Step 1 – Open Tinkercad

1. Open Google Chrome.
2. Visit https://www.tinkercad.com/
3. Sign in.
4. Select Circuits from the dashboard.
5. Click Create new Circuit.

Step 2 – Add components

From the Components panel, search and add:

1. Arduino Uno R3.
2. PIR Sensor.
3. LED.
4. Resistor.
5. Breadboard (optional, but useful for wiring).

Step 3 – Connect PIR sensor

1. PIR VCC → Arduino 5V.
2. PIR GND → Arduino GND.
3. PIR OUT → Arduino digital pin 2.

Step 4 – Connect LED

1. Place the LED on the breadboard.
2. Connect its longer leg (anode) to a 220Ω resistor.
3. Connect the other end of the resistor to Arduino digital pin 13.
4. Connect the shorter leg (cathode) to GND.

Step 5 – Open code editor

1. Click Code in the top-right corner.
2. Initially, the Blocks editor appears.
3. Click the dropdown that says Blocks.
4. Select Text.
5. Click Continue if prompted.
6. Remove existing code.
7. Paste the program.

6. COMPLETE ARDUINO PROGRAM

```
int pirPin = 2;
int ledPin = 13;
void setup() {
```

pinMode(pirPin, INPUT);

pinMode(ledPin, OUTPUT);

Serial.begin(9600);

```
}
void loop() {
```

int motionState = digitalRead(pirPin);

```
   if (motionState == HIGH) {
```
digitalWrite(ledPin, HIGH);

Serial.println("Motion detected!");

delay(1000);
```
   }
   else {
```
digitalWrite(ledPin, LOW);

Serial.println("No motion detected");
```
   }
 }
```
Note: This implementation uses digital pin 13 for the external LED. The manual describes pirPin and ledPin but does not explicitly specify the LED pin number in its written procedure. Pin 13 is the consistent choice used in this walkthrough.

7. CODE EXPLANATION

• int pirPin = 2; – Stores PIR sensor pin number.

• int ledPin = 13; – Stores LED pin number.

• void setup() – Executes once at startup.

• pinMode(pirPin, INPUT); – Configures PIR pin as input.

• pinMode(ledPin, OUTPUT); – Configures LED pin as output.

• Serial.begin(9600); – Starts serial communication.

• void loop() – Runs continuously.

• digitalRead(pirPin) – Reads the PIR sensor state.

• motionState == HIGH – Checks whether motion is detected.

• digitalWrite(ledPin, HIGH); – Turns ON the LED.

• Serial.println("Motion detected!"); – Displays motion message.

• digitalWrite(ledPin, LOW); – Turns OFF the LED.

• Serial.println("No motion detected"); – Displays no-motion message.

• delay(1000); – Pauses for one second.

8. PROGRAM LOGIC

Start Arduino → Read PIR sensor → Check whether motion is detected.

YES / HIGH → LED ON → Print “Motion detected!”

NO / LOW → LED OFF → Print “No motion detected”

The loop repeats continuously.

9. RUN SIMULATION AND CHECK OUTPUT

1. Click Start Simulation.

2. Wait for the PIR sensor to initialize.

3. Open Serial Monitor.

4. Observe messages.

5. Trigger motion using the PIR sensor’s simulation control, if available.

6. Check whether LED turns ON.

7. Stop triggering motion and observe whether LED turns OFF.

Example output:

No motion detected

No motion detected

No motion detected

Motion detected!

Motion detected!

No motion detected

No motion detected

Expected behavior:

• LED ON when motion is detected.

• LED OFF when no motion is detected.

A real PIR sensor can take time to stabilize after power-up. Its output may remain HIGH briefly after initialization.

10. EXPECTED RESULT

• PIR sensor detects motion.

• Arduino receives HIGH when motion is detected.

• LED turns ON.

• Arduino displays “Motion detected!” in Serial Monitor.

• When motion stops, LED turns OFF and “No motion detected” is displayed.

11. CONCLUSION

Thus, we successfully interfaced a PIR sensor with Arduino Uno using Tinkercad. The system detects motion and controls an LED accordingly, displaying the motion detection status on the Serial Monitor.

12. COMMON ERRORS AND SOLUTIONS

• LED not glowing – Check LED polarity and resistor connection.

• PIR not detecting motion – Check VCC, GND and OUT connections.

• No Serial Monitor output – Check Serial Monitor and 9600 baud rate.

• Compilation error – Check code syntax and braces.

• LED always ON – Check PIR output and sensor initialization.

• Simulation not starting – Check whether all components are connected properly.

13. IMPORTANT VIVA QUESTIONS AND ANSWERS

Q1. What is PIR?

PIR stands for Passive Infrared Sensor. It detects motion by sensing changes in infrared radiation.

Q2. What is the full form of PIR?

Passive Infrared.

Q3. What is the working principle of a PIR sensor?

It detects changes in infrared radiation emitted by objects such as humans and animals.

Q4. What are the three pins of a PIR sensor?

VCC – Power supply; GND – Ground; OUT – Output signal.

Q5. Why do we use Arduino Uno?

Arduino Uno acts as a microcontroller that reads the PIR sensor output and controls the LED.

Q6. What is the difference between HIGH and LOW?

HIGH represents digital logic 1; LOW represents digital logic 0.

Q7. What happens when motion is detected?

The PIR sensor sends HIGH to Arduino, and Arduino turns ON the LED.

Q8. What is digitalRead()?

An Arduino function used to read the digital state of a pin.

Q9. What is digitalWrite()?

It sets a digital output pin to HIGH or LOW.

Q10. Why do we use pinMode()?

It configures a pin as INPUT or OUTPUT.

Q11. What is Tinkercad?

An online platform used to design and simulate electronic circuits.

Q12. What are the applications of PIR sensors?

Automatic lighting systems, home security systems, motion detection alarms, automatic doors and intruder detection systems.

Q13. Difference between PIR and IR sensors?

A PIR sensor detects changes in infrared radiation emitted by objects, whereas an active IR sensor generally uses an infrared emitter and receiver to detect objects.

Q14. Why is a resistor connected with the LED?

To limit current and protect the LED from damage.

Q15. What is Serial.begin(9600)?

It initializes serial communication at a baud rate of 9600 bits per second.

14. EXECUTION CHECKLIST

■ Open Tinkercad and create a new circuit.

■ Add Arduino Uno R3.

■ Add PIR sensor.

■ Add LED and 220Ω resistor.

■ Connect PIR VCC to 5V.

■ Connect PIR GND to GND.

■ Connect PIR OUT to digital pin 2.

■ Connect LED to digital pin 13 through resistor.

■ Open Code → Blocks → Text.

■ Paste the Arduino program.

■ Start Simulation.

■ Open Serial Monitor.

■ Test motion detection and LED ON/OFF.

15. QUICK REVISION

• Sensor: PIR

• Software: Tinkercad

• Board: Arduino Uno

• PIR output pin: D2

• LED pin: D13

• PIR supply: 5V

• Motion detected: HIGH

• No motion: LOW

• Baud rate: 9600

EXPERIMENT 3 – DESIGN AND DEVELOP A USER INTERFACE FOR MONITORING AND CONTROLLING A CPS SYSTEM

Lab manual experiment 4.3 | Arduino IDE + ESP32 + Wi-Fi

1. AIM

Write a program to design and develop a user interface for monitoring and controlling a Cyber-Physical System (CPS).

What you are doing:

1. Connect two LEDs to ESP32.

2. Program ESP32 using Arduino IDE.

3. Connect ESP32 to a Wi-Fi network.

4. Create a web server on ESP32.

5. Access the server through its IP address.

6. Control two LEDs using ON/OFF buttons on a webpage.

2. REQUIREMENTS

Software:

• Arduino IDE

• ESP32 board support package

• Web browser (Chrome)

Hardware:

• ESP32 (ESP-WROOM-32)

• Micro USB data cable

• Breadboard

• Two LEDs

• Two 220Ω resistors

• Jumper wires

• Laptop/PC

3. UNDERSTAND CPS

CPS stands for Cyber-Physical System. It integrates physical components with computer-based systems for monitoring and controlling real-world processes.

Examples:

• Smart homes

• Industrial automation

• Autonomous vehicles

• Smart cities

• Healthcare monitoring systems

In this experiment:

• Physical system: LEDs connected to ESP32.

• Cyber system: ESP32 program and web interface.

• Communication: Wi-Fi.

• Control: ON/OFF buttons on the webpage.

When a button is pressed on the webpage, ESP32 receives the command and changes the physical LED state.

4. CIRCUIT CONNECTIONS

According to the lab manual:

• LED 1 positive/anode → GPIO 12 through 220Ω resistor.

• LED 1 negative/cathode → GND.

• LED 2 positive/anode → GPIO 14 through 220Ω resistor.

• LED 2 negative/cathode → GND.

Remember:

• First LED → GPIO 12.

• Second LED → GPIO 14.

• Both negative terminals → GND.

Use one resistor per LED.

5. EXACT PRACTICAL WORKFLOW

Step 1 – Connect ESP32 to laptop

1. Connect ESP32 to laptop using a USB data cable.

2. Check whether the power LED glows.

3. Open Arduino IDE.

Step 2 – Connect LEDs

1. Place two LEDs on the breadboard.

2. Connect positive terminal of first LED to GPIO 12 through a 220Ω resistor.

3. Connect its negative terminal to GND.

4. Connect positive terminal of second LED to GPIO 14 through another 220Ω resistor.

5. Connect its negative terminal to GND.

6. Double-check connections.

Step 3 – Configure Arduino IDE

1. Open Arduino IDE.

2. Go to Tools → Board → ESP32 Arduino.

3. Select ESP32 Dev Module (or the correct board available).

4. Go to Tools → Port.

5. Select the ESP32 COM port.

Step 4 – Prepare Wi-Fi details

Use:

const char* ssid = "Your_WiFi_Name";

const char* password = "Your_WiFi_Password";

Replace these values with the actual Wi-Fi network name and password. A mobile hotspot can also be used. Laptop/mobile and ESP32 must be on the same local network.

6. COMPLETE ARDUINO PROGRAM

The manual describes the ESP32 web server and GPIO 12/GPIO 14 controls. Its extracted text does not provide the complete source listing. The following complete implementation matches the functionality and URL structure described in the manual.

```
#include <WiFi.h>
#include <WebServer.h>
// WiFi credentials
```

const char* ssid = "Your_WiFi_Name";

const char* password = "Your_WiFi_Password";

```
// LED pins
const int led1 = 12;
const int led2 = 14;
// Create web server on port 80
WebServer server(80);
void handleRoot() {
  String webpage = "<html><body>";
webpage += "<h1>ESP32 LED Control</h1>";
webpage += "<h2>LED 1 (GPIO 12)</h2>";
webpage += "<a href='/12/on'><button>ON</button></a>";
webpage += "<a href='/12/off'><button>OFF</button></a>";
webpage += "<h2>LED 2 (GPIO 14)</h2>";
webpage += "<a href='/14/on'><button>ON</button></a>";
webpage += "<a href='/14/off'><button>OFF</button></a>";
webpage += "</body></html>";
server.send(200, "text/html", webpage);
}
void setup() {
Serial.begin(115200);
pinMode(led1, OUTPUT);
pinMode(led2, OUTPUT);
digitalWrite(led1, LOW);
digitalWrite(led2, LOW);
WiFi.begin(ssid, password);
Serial.print("Connecting to WiFi");
while (WiFi.status() != WL_CONNECTED) {
delay(500);
Serial.print(".");
}
Serial.println();
Serial.println("WiFi Connected!");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());
server.on("/", handleRoot);
server.on("/12/on", []() {
digitalWrite(led1, HIGH);
Serial.println("LED 1 ON");
server.sendHeader("Location", "/");
server.send(303);
});
server.on("/12/off", []() {
digitalWrite(led1, LOW);
Serial.println("LED 1 OFF");
server.sendHeader("Location", "/");
server.send(303);
});
server.on("/14/on", []() {
```

digitalWrite(led2, HIGH);

Serial.println("LED 2 ON");

server.sendHeader("Location", "/");

server.send(303);

```
  });
```

server.on("/14/off", []() {

digitalWrite(led2, LOW);

Serial.println("LED 2 OFF");

server.sendHeader("Location", "/");

server.send(303);

```
  });
```

server.begin();

Serial.println("Web Server Started");

```
}
void loop() {
```

server.handleClient();

```
}
```

Before uploading:

Change the SSID and password to the actual network credentials. Example only:

const char* ssid = "MyPhone";

const char* password = "12345678";

7. CODE EXPLANATION

• #include <WiFi.h> – Enables Wi-Fi functionality in ESP32.

• #include <WebServer.h> – Provides web server functionality.

• const char* ssid – Stores Wi-Fi network name.

• const char* password – Stores Wi-Fi password.

• const int led1 = 12 – Defines first LED pin.

• const int led2 = 14 – Defines second LED pin.

• WebServer server(80) – Creates web server on port 80.

• WiFi.begin() – Connects ESP32 to Wi-Fi.

• WiFi.status() – Checks Wi-Fi connection status.

• WiFi.localIP() – Returns ESP32's local IP address.

• server.on() – Defines how the server handles a URL request.

• server.begin() – Starts the web server.

• server.handleClient() – Processes incoming client requests.

• digitalWrite() – Controls LED ON/OFF state.

• server.send() – Sends an HTTP response.

• server.sendHeader() – Adds a response header.

• HTTP 303 redirect – Returns the browser to the main page after a button action.

8. IMPORTANT URLS

• /12/on – Turns ON LED 1.

• /12/off – Turns OFF LED 1.

• /14/on – Turns ON LED 2.

• /14/off – Turns OFF LED 2.

Example:

server.on("/12/on", []() {

digitalWrite(led1, HIGH);

```
});
```

Whenever a client requests /12/on, ESP32 sets GPIO 12 to HIGH and turns ON the first LED.

9. UPLOAD PROGRAM

1. Click Verify.

2. Wait for compilation.

3. Click Upload.

4. Wait until uploading finishes.

5. If the board gets stuck on Connecting, hold BOOT until upload starts.

6. Wait for ESP32 to restart.

10. FIND ESP32 IP ADDRESS

1. Open Tools → Serial Monitor.

2. Set baud rate to 115200.

3. Press EN/RESET on ESP32.

4. Wait for Wi-Fi connection.

Example output:

Connecting to WiFi....

WiFi Connected!

IP Address: 192.168.1.105

Web Server Started

The IP address will differ. Use the actual address displayed.

11. ACCESS THE ESP32 WEB SERVER

1. Connect laptop/mobile to the same Wi-Fi network as ESP32.

2. Open Chrome.

3. Enter the IP address, e.g. http://192.168.1.105

4. Press Enter.

5. The ESP32 LED Control webpage appears.

6. Click ON/OFF buttons for each LED.

Test:

• LED 1 ON → First LED glows.

• LED 1 OFF → First LED turns OFF.

• LED 2 ON → Second LED glows.

• LED 2 OFF → Second LED turns OFF.

The Serial Monitor displays the corresponding LED status messages.

12. COMPLETE WORKING

Laptop/mobile browser → User clicks ON/OFF → Wi-Fi communication → ESP32 web server receives HTTP request → ESP32 processes command → GPIO 12 or GPIO 14 changes state → Physical LED turns ON/OFF.

13. EXPECTED RESULT

• ESP32 connects to Wi-Fi.

• ESP32 creates a web server.

• IP address is displayed on Serial Monitor.

• Webpage opens through the IP address.

• Two LEDs can be controlled independently using ON/OFF buttons.

14. CONCLUSION

Thus, we successfully designed and developed a user interface for monitoring and controlling a Cyber-Physical System using ESP32. The LEDs connected to GPIO 12 and GPIO 14 were controlled through a web interface using Wi-Fi communication.

15. COMMON ERRORS AND SOLUTIONS

• COM port not showing – Check USB cable and driver.

• Compilation error: WiFi.h – Select an ESP32 board and install ESP32 board support.

• WebServer.h not found – Check ESP32 board package installation.

• Wi-Fi not connecting – Check SSID and password.

• IP address not appearing – Check Wi-Fi connection and reset ESP32.

• Webpage not opening – Ensure both devices are on the same network.

• LED not glowing – Check GPIO pin and LED polarity.

• Upload stuck at Connecting – Hold BOOT during upload initialization.

Important practical tip:

ESP32 generally requires a 2.4 GHz Wi-Fi network. Some college Wi-Fi networks have client isolation, which prevents the laptop from communicating with ESP32. If this happens, use a mobile hotspot if permitted.

16. IMPORTANT VIVA QUESTIONS AND ANSWERS

Q1. What is CPS?

CPS stands for Cyber-Physical System. It integrates physical components with computer-based systems for monitoring and controlling real-world processes.

Q2. What is ESP32?

ESP32 is a microcontroller developed by Espressif Systems that supports Wi-Fi and Bluetooth connectivity.

Q3. What is a web server?

A web server receives HTTP requests from clients and sends responses such as webpages.

Q4. What is the purpose of WiFi.h?

It provides Wi-Fi connectivity functions for ESP32.

Q5. What is the purpose of WebServer.h?

It provides functions to create and manage a web server on ESP32.

Q6. What is the purpose of WebServer server(80)?

It creates a web server that listens for HTTP requests on port 80.

Q7. What is an IP address?

An IP address is a network address used to identify a device and communicate with it over a network.

Q8. What is HTTP?

HTTP stands for Hypertext Transfer Protocol. It is used for communication between web clients and servers.

Q9. What is Wi-Fi?

Wi-Fi is a wireless networking technology that allows devices to communicate over a network.

Q10. Why do we use Serial Monitor?

To display debugging messages, Wi-Fi connection status and the ESP32 IP address.

Q11. What does server.begin() do?

It starts the web server and makes it ready to accept incoming requests.

Q12. What does server.handleClient() do?

It checks for and processes incoming client requests.

Q13. What is GPIO?

GPIO stands for General Purpose Input/Output. It is used to connect and control external devices.

Q14. What is port 80?

Port 80 is the standard port used for HTTP communication.

Q15. Difference between web server and web browser?

A web server receives requests and provides responses. A web browser sends requests and displays webpages.

Q16. What is the difference between digital HIGH and LOW?

HIGH activates the output, while LOW deactivates it. In this experiment, HIGH turns ON the LED and LOW turns it OFF.

Q17. Why is this an example of IoT?

Because ESP32 communicates over Wi-Fi and allows physical devices to be monitored and controlled through a network interface.

17. EXECUTION CHECKLIST

■ Connect ESP32 to laptop.

■ Connect LED 1 to GPIO 12 through 220Ω resistor.

■ Connect LED 2 to GPIO 14 through 220Ω resistor.

■ Connect both LED negative terminals to GND.

■ Open Arduino IDE.

■ Select ESP32 Dev Module and COM port.

■ Enter Wi-Fi SSID and password in code.

■ Paste the complete program.

■ Verify and upload code.

■ Open Serial Monitor at 115200 baud.

■ Find ESP32 IP address.

■ Connect laptop/mobile to the same Wi-Fi.

■ Open IP address in Chrome.

■ Test LED 1 ON/OFF.

■ Test LED 2 ON/OFF.

18. QUICK REVISION

• Microcontroller: ESP32.

• LED pins: GPIO 12 and GPIO 14.

• Libraries: WiFi.h and WebServer.h.

• Web server: Port 80.

• IP address: Obtain from Serial Monitor at 115200 baud.

• Four URLs: /12/on, /12/off, /14/on, /14/off.

EXPERIMENT 4 – SENDING SENSOR DATA AND DISPLAYING IT ON A WEBPAGE

Lab manual experiment 4.4 | Arduino IDE + ESP32 + Wi-Fi

1. AIM

Write a program for sending sensor data to the cloud and displaying it on a webpage.

What you are doing:

1. Connect a 10K potentiometer to ESP32.

2. Connect ESP32 to Wi-Fi.

3. Read analog values from the potentiometer.

4. Create a web server using ESP32.

5. Display the potentiometer reading on a webpage.

6. Rotate the potentiometer and observe changing values.

2. REQUIREMENTS

Software:

• Arduino IDE

• ESP32 board support package

• Web browser (Google Chrome)

Hardware:

• ESP32 (ESP-WROOM-32)

• 10K potentiometer

• Breadboard

• Micro USB data cable

• Male-to-female jumper wires

• Laptop/PC

3. CIRCUIT CONNECTIONS

• Middle potentiometer pin → GPIO 34.

• Red/outer pin → 3.3V.

• Orange/outer pin → GND.

Remember:

• Middle → GPIO 34.

• Outer → 3.3V.

• Outer → GND.

Use 3.3V, not 5V.

4. EXACT PRACTICAL WORKFLOW

Step 1 – Connect ESP32 to laptop

1. Connect ESP32 to laptop using a USB data cable.

2. Check whether the power LED glows.

3. Open Arduino IDE.

Step 2 – Connect potentiometer

1. Place potentiometer on breadboard.

2. Connect middle pin to GPIO 34.

3. Connect one outer pin to 3.3V.

4. Connect other outer pin to GND.

5. Check all connections.

Step 3 – Configure Arduino IDE

1. Open Arduino IDE.

2. Go to Tools → Board → ESP32 Arduino.

3. Select ESP32 Dev Module.

4. Go to Tools → Port.

5. Select correct COM port.

Step 4 – Configure Wi-Fi

Use:

const char* ssid = "Your_WiFi_Name";

const char* password = "Your_WiFi_Password";

Replace these with actual Wi-Fi credentials. Use a 2.4 GHz network or mobile hotspot if required.

5. COMPLETE ARDUINO PROGRAM

The manual describes using WiFi.h and WebServer.h and a web server that displays the potentiometer reading. The following is the complete program used in this walkthrough.

```
#include <WiFi.h>
#include <WebServer.h>
// WiFi credentials
```

const char* ssid = "Your_WiFi_Name";

const char* password = "Your_WiFi_Password";

```
// Potentiometer pin
const int potPin = 34;
```

int potValue = 0;

```
// Create web server on port 80
WebServer server(80);
// Function to display sensor value
void handleRoot() {
```

potValue = analogRead(potPin);

```
  String webpage = "<html><body>";
```

webpage += "<h1>ESP32 Sensor Monitoring</h1>";

webpage += "<h2>Potentiometer Value: ";

webpage += String(potValue);

webpage += "</h2>";

webpage += "</body></html>";

server.send(200, "text/html", webpage);

```
}
void setup() {
```

Serial.begin(115200);

pinMode(potPin, INPUT);

WiFi.begin(ssid, password);

Serial.print("Connecting to WiFi");

while (WiFi.status() != WL_CONNECTED) {

```
delay(500);
Serial.print(".");
  }
Serial.println();
Serial.println("WiFi Connected!");
Serial.print("IP Address: ");
Serial.println(WiFi.localIP());
server.on("/", HTTP_GET, handleRoot);
server.begin();
Serial.println("Web Server Started");
}
void loop() {
potValue = analogRead(potPin);
server.handleClient();
delay(100);
}
```

Before uploading:

Replace Wi-Fi name and password with actual credentials.

6. CODE EXPLANATION

• #include <WiFi.h> – Enables Wi-Fi functionality.

• #include <WebServer.h> – Provides web server functionality.

• ssid – Stores Wi-Fi network name.

• password – Stores Wi-Fi password.

• potPin = 34 – Defines potentiometer input pin.

• potValue – Stores analog sensor reading.

• WebServer server(80) – Creates web server on port 80.

• analogRead(potPin) – Reads potentiometer value.

• WiFi.begin() – Connects ESP32 to Wi-Fi.

• WiFi.localIP() – Gets ESP32 IP address.

• server.on() – Defines webpage request handler.

• server.begin() – Starts web server.

• server.handleClient() – Processes incoming browser requests.

• server.send() – Sends webpage response to browser.

7. THREE IMPORTANT FUNCTIONS

analogRead():

```
potValue = analogRead(potPin);
```

Reads analog value from GPIO 34. ESP32 reading ranges from 0 to 4095.

handleRoot():

```
void handleRoot() {
potValue = analogRead(potPin);
}
```

This function handles the main webpage request and displays the current sensor value.

server.handleClient():

```
server.handleClient();
```

Checks for incoming browser requests and processes them.

8. UPLOAD PROGRAM

1. Click Verify.

2. Wait for compilation.

3. Click Upload.

4. Wait until upload finishes.

5. Press ESP32 EN/RESET if required.

If uploading gets stuck at Connecting, hold BOOT until uploading starts.

9. FIND ESP32 IP ADDRESS

1. Open Tools → Serial Monitor.

2. Select baud rate 115200.

3. Press ESP32 RESET.

4. Wait for Wi-Fi connection.

Example:

Connecting to WiFi....

WiFi Connected!

IP Address: 192.168.1.105

Web Server Started

Your IP address will differ. Note the actual address.

10. OPEN THE WEBPAGE

1. Open Google Chrome.

2. Enter the ESP32 IP address in the address bar, e.g. http://192.168.1.105

3. Press Enter.

4. Webpage displays the potentiometer reading.

Example webpage:

ESP32 Sensor Monitoring

Potentiometer Value: 2233

This is an example only; actual value varies.

11. TEST THE POTENTIOMETER

1. Keep webpage open.

2. Rotate potentiometer slowly.

3. Refresh webpage.

4. Observe new sensor reading.

5. Rotate in opposite direction and observe the change.

Values should change approximately between 0 and 4095.

Important practical detail:

The supplied program updates the value when the browser requests the page. Refresh the webpage after rotating the potentiometer to see the latest reading. It does not automatically refresh continuously.

12. COMPLETE WORKING

Potentiometer → Variable analog voltage → ESP32 reads through GPIO 34 → Wi-Fi web server processes HTTP request → Webpage displays potentiometer reading.

13. EXPECTED RESULT

• ESP32 connects to Wi-Fi.

• A web server is created.

• IP address is displayed in Serial Monitor.

• Webpage displays potentiometer reading.

• Rotating potentiometer changes displayed reading after refreshing the page.

14. CONCLUSION

Thus, we successfully interfaced a potentiometer with ESP32 and developed a web server to read analog sensor values and display them on a webpage using Wi-Fi communication.

15. IMPORTANT CLARIFICATION ABOUT “CLOUD”

The lab manual calls this experiment sending data to the cloud. However, the actual procedure and code described in pages 21–23 create a web server directly on ESP32 and display data using its local IP address. This implementation demonstrates local network monitoring, not uploading sensor readings to an external cloud platform such as AWS or ThingSpeak. For the practical, follow the ESP32 web-server procedure in the manual.

16. COMMON ERRORS AND SOLUTIONS

• Wi-Fi not connecting – Check SSID and password.

• IP address not appearing – Check Wi-Fi and restart ESP32.

• Webpage not opening – Ensure laptop and ESP32 are on same network.

• Value always 0 – Check GPIO 34 and potentiometer wiring.

• Value not changing – Check middle-pin connection.

• WiFi.h not found – Install/select ESP32 board package.

• WebServer.h not found – Check ESP32 board package installation.

• Webpage shows old value – Refresh browser.

Library note:

The manual mentions WebServer_ESP32_ENC. The code above uses standard WebServer.h included with the ESP32 Arduino board package. If the examiner provides a specific library or starter code, follow that lab setup.

17. IMPORTANT VIVA QUESTIONS AND ANSWERS

Q1. What is the aim of this experiment?

To send sensor data using ESP32 and display sensor readings on a webpage.

Q2. Which sensor is used?

A 10K potentiometer is used as an analog input device.

Q3. Which GPIO pin is used?

GPIO 34.

Q4. What is the range of ESP32 analog readings?

0 to 4095 (12-bit ADC).

Q5. Which libraries are used?

WiFi.h and WebServer.h.

Q6. What is a web server?

A web server receives requests from clients and sends responses such as webpages.

Q7. What is the purpose of WebServer server(80)?

It creates a web server that listens on HTTP port 80.

Q8. What is the purpose of WiFi.begin()?

It initiates Wi-Fi connection using the given SSID and password.

Q9. What is the purpose of WiFi.localIP()?

It returns the local IP address assigned to ESP32.

Q10. What is the purpose of server.on()?

It defines which function should execute when a particular URL is requested.

Q11. What is the purpose of server.handleClient()?

It processes incoming requests from web browsers.

Q12. What is HTTP?

HTTP stands for Hypertext Transfer Protocol. It is used for communication between web browsers and servers.

Q13. Difference between analog and digital sensors?

Analog sensors produce continuously varying signals, while digital sensors provide discrete digital outputs.

Q14. Why do we use GPIO 34?

It is an ADC-capable input pin that can read analog signals from the potentiometer.

Q15. What is the difference between Experiment 3 and Experiment 4?

Experiment 3 controls LEDs through a webpage, whereas Experiment 4 reads potentiometer values and displays them on a webpage.

Q16. What is the function of server.send()?

It sends an HTTP response from ESP32 to the connected client.

18. EXECUTION CHECKLIST

■ Connect ESP32 to laptop.

■ Connect potentiometer middle pin to GPIO 34.

■ Connect outer pins to 3.3V and GND.

■ Open Arduino IDE.

■ Select ESP32 board and COM port.

■ Enter Wi-Fi SSID and password.

■ Paste complete program.

■ Verify and upload code.

■ Open Serial Monitor at 115200 baud.

■ Find ESP32 IP address.

■ Open IP address in Chrome.

■ Observe potentiometer reading.

■ Rotate potentiometer and refresh webpage.

■ Verify readings are changing.

19. QUICK REVISION

• Potentiometer middle pin → GPIO 34.

• ESP32 ADC range → 0–4095.

• Libraries → WiFi.h and WebServer.h.

• Web server → Port 80.

• IP address → Obtain from Serial Monitor at 115200 baud.

• Refresh browser to see updated reading in this supplied implementation.

EXPERIMENT 5 – INDUSTRIAL DATA ANALYSIS USING EXCEL

Lab manual experiment 4.5 | Microsoft Excel

1. AIM

Write a program for performing industrial data analysis using relevant tools and techniques.

What you are doing:

1. Obtain an IoT dataset from Kaggle.

2. Open dataset in Microsoft Excel.

3. Analyse temperature readings for 24 hours.

4. Analyse temperature readings for 48 hours.

5. Create graphs using Excel.

6. Repeat analysis for humidity readings.

7. Interpret graphs and write observations.

2. REQUIREMENTS

Software:

• Microsoft Excel

• Internet connection for downloading dataset

Dataset:

• Farm Temperature and Humidity IoT dataset from Kaggle.

According to the lab manual:

• Dataset name: Farm Temperature and Humidity.

• File size: Approximately 86 KB.

• Data duration: 3 months.

• Number of farms: 4.

• Distance between farms: Approximately 1 km.

• Reading frequency: Every 1 hour.

• Main parameters: Temperature and Humidity.

3. UNDERSTAND THE DATASET

The dataset contains readings collected using sensors from four different farms.

Temperature:

Temperature indicates how hot or cold the environment is.

Humidity:

Humidity indicates the amount of moisture present in the air.

Readings are collected every hour, allowing daily and repeated patterns to be studied.

Illustrative dataset structure (values below are examples only, not actual Kaggle readings):

Time | Temperature | Humidity

00:00 | 24.5 | 75

01:00 | 23.8 | 78

02:00 | 23.2 | 81

03:00 | 22.9 | 83

04:00 | 22.5 | 85

...

23:00 | 29.5 | 61

Actual dataset may have different column names and multiple farm readings. Identify the appropriate temperature and humidity columns.

4. EXACT PRACTICAL WORKFLOW

Step 1 – Download dataset

1. Open Google Chrome.

2. Visit https://www.kaggle.com/datasets

3. Search for “Farm Temperature and Humidity”.

4. Find the dataset matching the lab manual.

5. Download dataset.

6. Extract ZIP file if necessary.

7. Locate Excel or CSV file.

If examiner has already provided the dataset, use that instead.

Step 2 – Open dataset in Excel

1. Open Microsoft Excel.

2. Click File → Open → Browse.

3. Select downloaded dataset.

4. Click Open.

5. Observe columns containing temperature and humidity readings.

Step 3 – Temperature analysis for 24 hours

1. Find the temperature column.

2. Select the first 24 hourly readings for one day.

3. Include corresponding time/hour column so Excel can use it as horizontal axis.

4. Include column headings if possible.

5. Go to Insert.

6. Click Recommended Charts.

7. Select a Line Chart.

8. Click OK.

9. Observe how temperature changes throughout the day.

Expected observation according to lab manual:

• Temperature increases during morning.

• It reaches a peak around 12 PM.

• After the peak, temperature starts decreasing.

• At night, temperature falls.

Journal observation:

The 24-hour temperature graph shows that temperature increases during the morning and reaches its maximum around 12 noon. After that, temperature gradually decreases, and lower temperatures are observed at night.

Step 4 – Temperature analysis for 48 hours

1. Return to original dataset.

2. Select temperature column.

3. Select 48 hourly readings (two days).

4. Include corresponding time/hour values.

5. Click Insert.

6. Select Recommended Charts.

7. Choose a Line Chart.

8. Click OK.

Expected observation according to lab manual:

• First day temperature reaches a peak around hour 12.

• Temperature decreases after that.

• A minimum is observed around hour 24 in the manual's interpretation.

• On second day, another peak appears around hour 36 (24 + 12).

Why hour 36?

24 + 12 = 36.

It represents 12 noon on the second day when counting from hour zero on the first day.

Journal observation:

The 48-hour temperature graph shows a repeated daily pattern. Temperature reaches a peak around hour 12 on the first day and around hour 36 on the second day. This indicates a similar daily temperature variation.

Step 5 – Humidity analysis for 24 hours

1. Return to original dataset.

2. Find humidity column.

3. Select first 24 hourly readings.

4. Include corresponding time values.

5. Go to Insert.

6. Click Recommended Charts.

7. Select Scatter Chart.

8. Click OK.

Expected observation according to lab manual:

• Humidity increases during nighttime.

• Humidity changes throughout the day.

• Scatter graph helps visualize the relationship between time and humidity readings.

Journal observation:

The 24-hour humidity scatter graph shows that humidity generally increases during nighttime. The graph helps us understand the variation in humidity throughout the day.

Step 6 – Humidity analysis for 48 hours

1. Select humidity column.

2. Select 48 hourly readings.

3. Include corresponding time/hour values.

4. Click Insert.

5. Select Scatter Chart.

6. Choose Scatter with markers.

7. Click OK.

Expected observation according to lab manual:

• Humidity follows a cyclic pattern over two days.

• A noticeable fall in humidity occurs around hour 13 (1 PM).

• A similar fall occurs around hour 37 (24 + 13), on the second day.

Journal observation:

The 48-hour humidity graph shows a cyclic pattern. Humidity decreases around hour 13 on the first day and around hour 37 on the second day. This indicates a similar humidity variation across both days.

5. SUMMARY OF FOUR GRAPHS

Graph 1 – Temperature (24 hours):

Chart: Line Chart.

Observation: Temperature peaks around 12 noon and falls at night.

Graph 2 – Temperature (48 hours):

Chart: Line Chart.

Observation: Peaks around hour 12 and hour 36.

Graph 3 – Humidity (24 hours):

Chart: Scatter Chart.

Observation: Humidity increases at night.

Graph 4 – Humidity (48 hours):

Chart: Scatter Chart.

Observation: Cyclic pattern with falls around hours 13 and 37.

These are expected patterns described in the manual. Actual observations should be verified against the dataset and graphs generated.

6. IMPORTANT EXCEL OPERATIONS

• Open dataset – File → Open.

• Select data – Click and drag cells.

• Insert chart – Insert → Recommended Charts.

• Create line graph – Insert → Line Chart.

• Create scatter graph – Insert → Scatter.

• Change chart title – Click Chart Title.

• Add axis titles – Chart Design → Add Chart Element → Axis Titles.

• Save workbook – Ctrl + S.

• Save as Excel file – File → Save As → .xlsx.

Recommended chart titles:

1. Temperature Analysis – 24 Hours.

2. Temperature Analysis – 48 Hours.

3. Humidity Analysis – 24 Hours.

4. Humidity Analysis – 48 Hours.

Recommended axis labels:

• X-axis: Time (Hours).

• Y-axis: Temperature or Humidity, according to the graph.

7. COMMON MISTAKES AND SOLUTIONS

• Graph not showing correctly – Select correct data range.

• Time not appearing on X-axis – Include time column when selecting data.

• Wrong graph generated – Select Line or Scatter manually.

• Graph shows only one point – Check that multiple numeric readings are selected.

• Dataset opens incorrectly – Import CSV using Excel Data → From Text/CSV.

• Values treated as text – Convert cells into numeric format.

• Graph difficult to understand – Add chart title and axis labels.

8. EXPECTED RESULT

• Successfully imported Farm Temperature and Humidity dataset into Excel.

• Analysed temperature data for 24 and 48 hours.

• Analysed humidity data for 24 and 48 hours.

• Created line charts and scatter charts.

• Identified daily patterns and fluctuations in temperature and humidity.

9. CONCLUSION

Thus, we successfully performed Industrial IoT Data Analysis using Microsoft Excel. We analysed farm temperature and humidity readings for 24 and 48 hours using line charts and scatter plots and identified patterns and variations in the collected sensor data.

10. IMPORTANT VIVA QUESTIONS AND ANSWERS

Q1. What is IoT?

IoT stands for Internet of Things. It is a network of interconnected devices that collect and exchange data.

Q2. What is IoT data analytics?

It is the process of collecting, processing, analysing and interpreting data generated by IoT devices to obtain useful insights.

Q3. Which software is used in this experiment?

Microsoft Excel.

Q4. Which dataset is used?

Farm Temperature and Humidity dataset downloaded from Kaggle.

Q5. What is the duration of the dataset?

Three months.

Q6. How many farms are included?

Four farms.

Q7. How frequently are readings collected?

Every one hour.

Q8. What is temperature?

Temperature measures the degree of hotness or coldness of an environment.

Q9. What is humidity?

Humidity is the amount of water vapour present in the air.

Q10. What is data visualization?

Graphical representation of data using charts, graphs and dashboards.

Q11. Why do we use line charts?

Line charts help visualize trends and changes in data over time.

Q12. Why do we use scatter plots?

Scatter plots help visualize the distribution and relationship between two numerical variables.

Q13. Difference between line chart and scatter plot?

A line chart connects data points to show trends over time. A scatter plot displays individual points to show distribution and relationships.

Q14. What are the four main components of IoT data analytics mentioned in the manual?

1. Data Collection.

2. Data Storage.

3. Data Processing.

4. Data Analysis.

The manual also separately explains Data Visualization as an important part of presenting insights.

Q15. What is data preprocessing?

It involves preparing raw data for analysis through cleaning, transformation and normalization.

Q16. What is the use of Kaggle?

Kaggle is an online platform that provides datasets, machine learning resources and data science competitions.

Q17. What did you observe in the 24-hour temperature graph?

Temperature reaches a peak around 12 noon and decreases at night.

Q18. What did you observe in the 48-hour temperature graph?

Peaks occur around hour 12 and hour 36.

Q19. What did you observe in the humidity graph?

Humidity generally increases at night, follows a cyclic pattern and shows falls around hours 13 and 37 in the 48-hour analysis.

11. EXECUTION CHECKLIST

■ Open Kaggle and download Farm Temperature and Humidity dataset (if not provided).

■ Open dataset in Microsoft Excel.

■ Identify temperature and humidity columns.

■ Select 24 hours of temperature data.

■ Insert a line chart.

■ Interpret 24-hour temperature graph.

■ Select 48 hours of temperature data.

■ Insert second line chart.

■ Interpret 48-hour temperature graph.

■ Select 24 hours of humidity data.

■ Insert a scatter chart.

■ Select 48 hours of humidity data.

■ Insert another scatter chart.

■ Write observations for all four graphs.

■ Save Excel workbook.

12. QUICK REVISION

• Software: Microsoft Excel.

• Dataset: Farm Temperature and Humidity.

• Farms: 4.

• Duration: 3 months.

• Reading interval: 1 hour.

• Temperature charts: Line charts.

• Humidity charts: Scatter charts.

• Temperature peaks: Hours 12 and 36.

• Humidity falls: Hours 13 and 37.

FINAL REVISION SHEET – ALL FIVE EXPERIMENTS

Experiment 1 – Potentiometer with ESP32

• Main component/software: ESP32 + potentiometer + Arduino IDE.

• GPIO: 34.

• ADC range: 0–4095.

• Alert: warning when value > 2000.

• Serial baud: 115200.

• Delay: 500 ms.

Experiment 2 – PIR Sensor

• Main component/software: Arduino Uno + PIR + Tinkercad.

• PIR output: D2.

• LED: D13.

• PIR supply: 5V.

• Motion detected: HIGH.

• No motion: LOW.

• Serial baud: 9600.

Experiment 3 – CPS Web Interface

• Main component/software: ESP32 + two LEDs + Arduino IDE.

• LED pins: GPIO 12 and GPIO 14.

• Libraries: WiFi.h and WebServer.h.

• Web server: Port 80.

• IP address: Serial Monitor at 115200 baud.

• URLs: /12/on, /12/off, /14/on, /14/off.

Experiment 4 – Sensor Data on Webpage

• Main component/software: ESP32 + potentiometer + Arduino IDE.

• Potentiometer middle pin: GPIO 34.

• Supply: 3.3V and GND.

• ADC range: 0–4095.

• Libraries: WiFi.h and WebServer.h.

• Web server: Port 80.

• IP address: Serial Monitor at 115200 baud.

• Refresh browser to see updated reading in the supplied implementation.

Experiment 5 – Industrial Data Analysis

• Main software: Microsoft Excel.

• Dataset: Farm Temperature and Humidity.

• Farms: 4.

• Duration: 3 months.

• Reading frequency: 1 hour.

• Temperature: Line charts, 24 and 48 hours.

• Humidity: Scatter charts, 24 and 48 hours.

• Temperature peaks: Hours 12 and 36.

• Humidity falls: Hours 13 and 37.

RECOMMENDED PREPARATION ORDER FOR THE NIGHT BEFORE THE EXAM

1. Experiment 1: Practise writing the small Arduino program.

2. Experiment 2: Memorize circuit connections and PIR code.

3. Experiment 3: Focus on Wi-Fi, IP address, web server and LED control.

4. Experiment 4: Understand potentiometer-to-webpage workflow and code.

5. Experiment 5: Practise creating at least one line chart and one scatter chart in Excel.

Important preparation suggestion:

Practise Experiments 3 and 4 together because they share ESP32, Wi-Fi setup, COM port, IP address and web server concepts.

END OF COMPLETE PREPARATION GUIDE